TECHNICAL DOCUMENTATION — CONFIDENTIAL

SignalRisk

Integration & Architecture Reference

Complete technical guide for integrating, deploying, and operating the SignalRisk real-time fraud decision platform. For engineering teams, platform architects, and enterprise buyers.

13 Microservices | NestJS + TypeScript | Kafka Event Pipeline | RS256 JWT Auth | Neo4j Graph Intel | Kubernetes Ready | OpenAPI Documented

Version 0.1.0 | March 2026 | SignalRisk Platform

Table of Contents

OVERVIEW

1 Executive Summary & Platform Overview 3

2 System Architecture 4

3 Service Catalogue — All 13 Microservices 5

INTEGRATION GUIDE

4 Authentication & API Key Setup 7

5 SDK Integration (Web & Mobile) 8

6 Webhook Integration 10

7 API Reference 11

DECISION ENGINE

8 Decision Engine & Rule DSL 13

9 Feedback Loop & Rule Weight Tuning 15

OPERATIONS

10 Observability & Error Tracking 16

11 Deployment Architecture 18

12 Security Model 19

13 Performance SLAs & Capacity 20

14 Go-Live Checklist 21

1 Executive Summary

What SignalRisk does and how it fits into your payment stack

SignalRisk is a **real-time fraud decision engine** built for payment processors, fintech platforms, and marketplaces that need sub-100ms fraud scoring on every transaction. It evaluates five independent intelligence signals in parallel, runs a configurable DSL rule engine, and emits a deterministic ALLOW / REVIEW / BLOCK decision — all within a single synchronous API call from your checkout flow.

For Engineers

- Single REST endpoint per event submission
- JavaScript/TypeScript SDK (npm package)
- Webhook callbacks for async decisions
- OpenAPI spec for every service

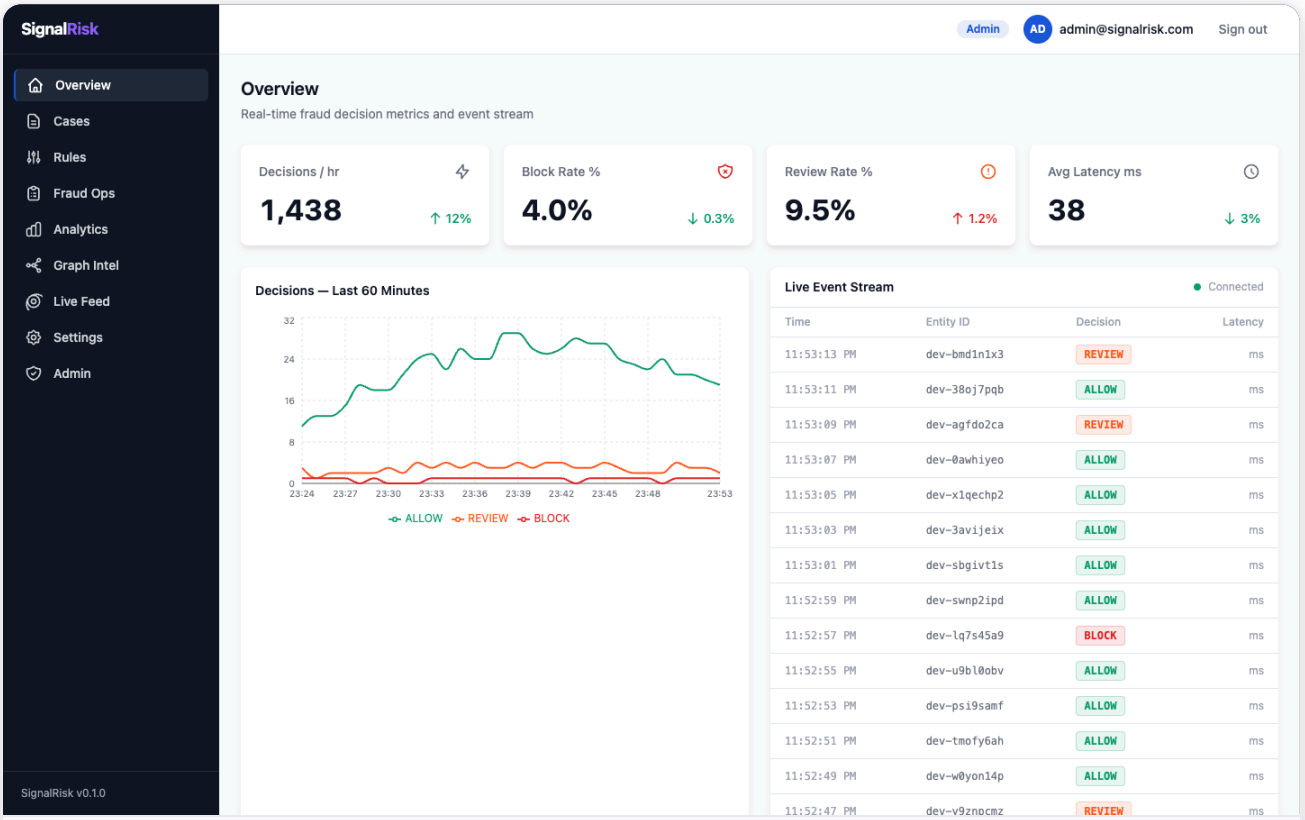
For Fraud Teams

- Analyst dashboard for case review
- No-code rule authoring via DSL
- Automated feedback loop improves accuracy
- SLA-tracked case management queue

For Platform Teams

- Full Kubernetes Helm chart included
- GitOps-ready with ArgoCD config
- OpenTelemetry traces on all services
- Prometheus metrics + Grafana dashboards

Platform Dashboard



Real-time Overview dashboard — KPI cards, 60-minute decision trend, live event stream

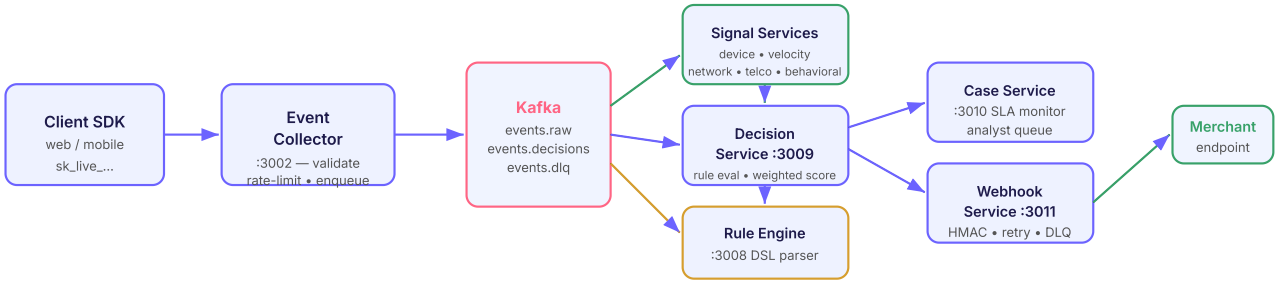
Key Metrics

Metric	Value	Notes
P99 decision latency	< 100ms	End-to-end, all 5 signals evaluated in parallel
Signal timeout	150ms	AbortController per signal; degraded signals default to neutral score
Throughput	5,000 req/sec	Sustained under k6 load test; horizontally scalable
JWT token TTL	15 minutes	Auto-renewed; RS256 signed
Webhook retry	3 attempts	Exponential backoff: 1m → 5m → 30m
Case SLA — BLOCK	4 hours	Dashboard highlights breached cases in red
Case SLA — REVIEW	24 hours	Monitored every 5 minutes by SlaMonitorService
Test coverage	840+ tests	Unit + integration + E2E across all services

2 System Architecture

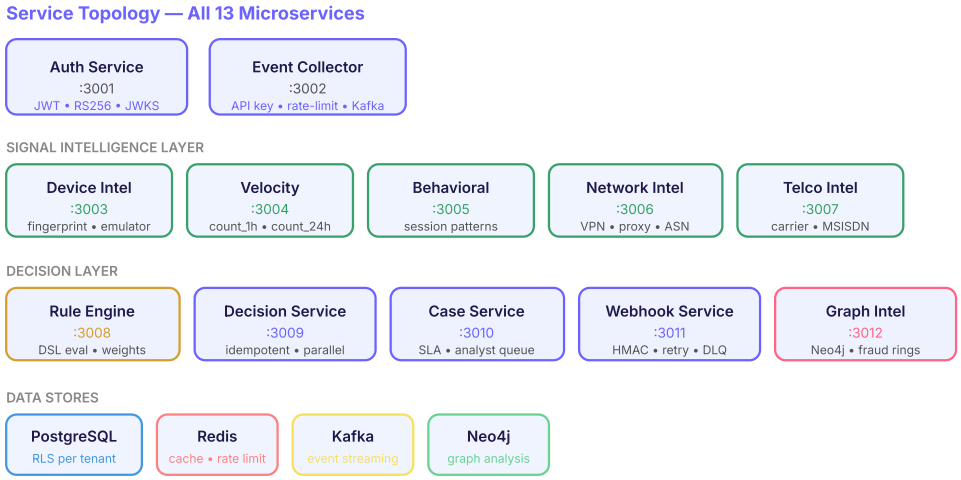
Event pipeline, service topology, and data flow

Event Pipeline



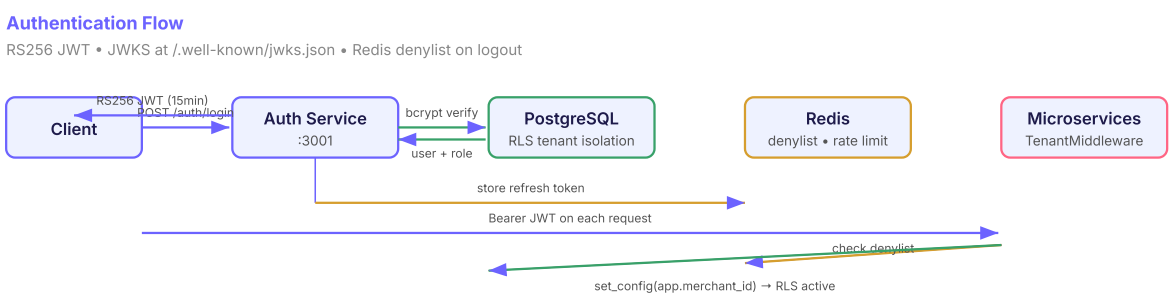
Event pipeline: Client SDK → Event Collector → Kafka → Signal Services + Decision Engine → Case & Webhook Services

Service Topology



All 13 microservices with ports, roles, and data store dependencies

Authentication Flow



RS256 JWT authentication flow with Redis denylist and PostgreSQL RLS tenant isolation

Design Principles

Tenant Isolation

Every database query executes with `SELECT set_config('app.merchant_id', $1, true)` before accessing tables. PostgreSQL Row-Level Security policies enforce that merchants can only read their own data — even if a query bug omits a WHERE clause.

Idempotency

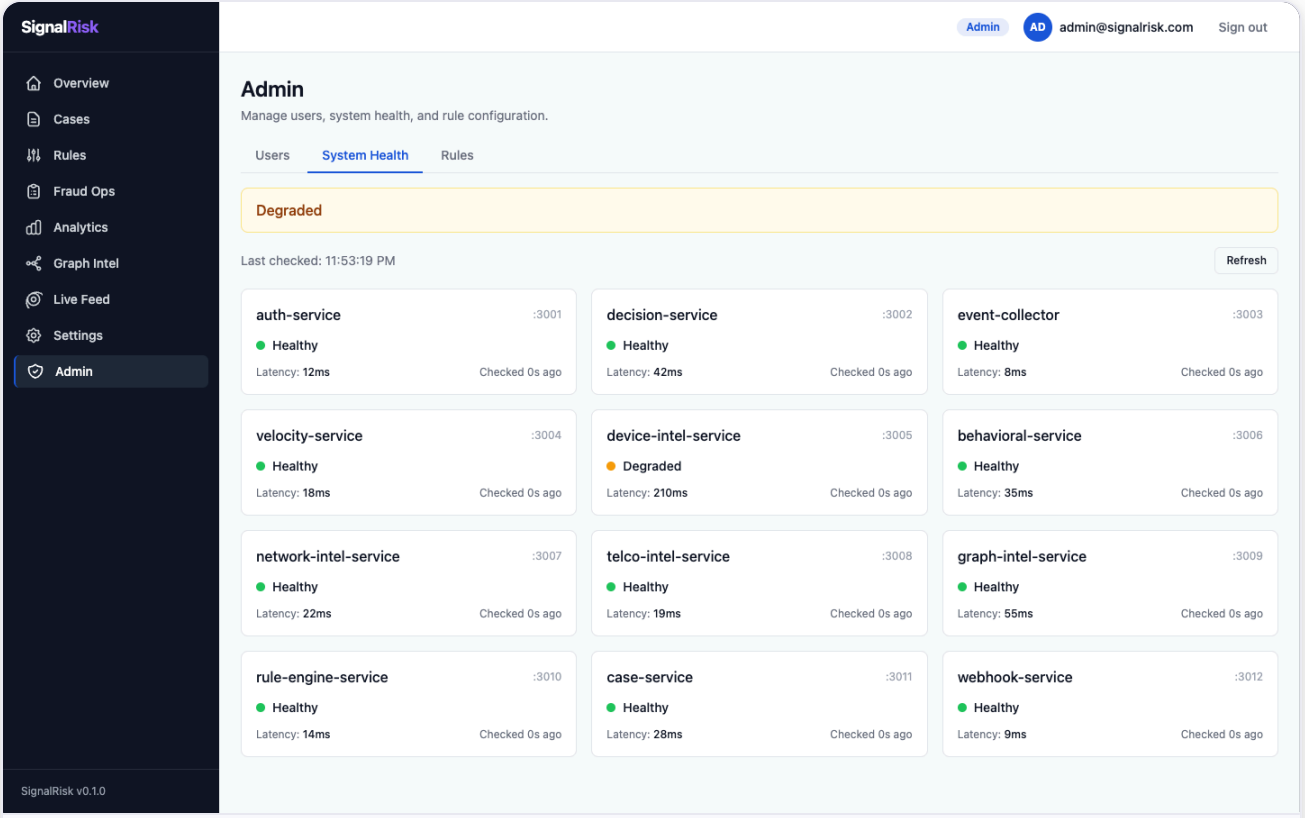
The Decision Service stores a hash of every `eventId` in Redis with a 24-hour TTL. Duplicate submissions return the original decision without re-running the pipeline. This ensures safe retry from mobile SDKs.

3 Service Catalogue

Responsibilities, ports, and dependencies of all 13 microservices

Service	Port	Responsibility	Stores
auth-service	`3001`	JWT login/refresh/logout, merchant onboarding, JWKS endpoint, API key management	PostgreSQL, Redis
event-collector	`3002`	HTTP ingestion, API key validation, Redis rate-limiting, Kafka producer	Redis, Kafka
device-intel-service	`3003`	Device fingerprinting, emulator detection, trust score (weight 0.35)	Redis (cache)
velocity-service	`3004`	Transaction velocity per entity: count_1h, count_24h, amount_1h (weight 0.25)	Redis (sliding window)
behavioral-service	`3005`	Session pattern analysis, typing cadence, interaction anomalies (weight 0.20)	PostgreSQL
network-intel-service	`3006`	VPN/proxy detection, ASN lookup, geo-mismatch scoring (weight 0.15)	Redis (cache)
telco-intel-service	`3007`	Phone number validation, carrier risk score, MSISDN verification (weight 0.05)	Redis (cache)
rule-engine-service	`3008`	DSL parser + evaluator, rule CRUD API, weight storage in Redis	PostgreSQL, Redis
decision-service	`3009`	Parallel signal fetch, rule evaluation, score aggregation, idempotent decisions	PostgreSQL, Redis
case-service	`3010`	Case creation, analyst assignment, resolution workflow, SLA monitoring every 5min	PostgreSQL
webhook-service	`3011`	HMAC-signed delivery, exponential retry, DLQ, outbox pattern	PostgreSQL, Kafka
graph-intel-service	`3012`	Neo4j fraud ring detection, entity relationship traversal, chargeback propagation	Neo4j
dashboard (UI)	`5173`	React SPA — analyst dashboard for case management, rule authoring, real-time monitoring	—

System Health Dashboard



Admin → System Health tab — live status of all 13 services with response times and last-checked timestamps

4 Authentication & API Key Setup

JWT authentication for dashboard users; API key authentication for server-side event submission

Dashboard Authentication (JWT)

The dashboard and internal service calls use RS256 JWT tokens. Tokens are issued by `auth-service:3001` and verified locally by each service using the public JWKS.

Endpoint	Description
`POST /auth/login`	Issue access + refresh token
`POST /auth/refresh`	Rotate refresh token
`POST /auth/logout`	Add token to Redis denylist
`GET /.well-known/jwks.json`	Public key for verification

API Key Authentication (Events)

Server-side event submission uses API keys. Keys are in the format `sk_live_<32hex>`. Only the bcrypt hash of the key is stored in the database — the plaintext is shown once on creation.

```
// HTTP header on all event requests
Authorization: Bearer sk_live_a1b2c3...
```

The first 8 characters (prefix) identify the key for fast lookup. The Event Collector validates and hashes on each request.

Creating an API Key

```
// 1. Login to get JWT
POST /auth/login
{
  "email": "admin@yourcompany.com",
  "password": "..."
}
// → { accessToken, refreshToken, user }

// 2. Create API key (use accessToken in Authorization header)
POST /merchants/{merchantId}/api-keys
Authorization: Bearer <accessToken>
{
  "name": "production-checkout",
  "scopes": ["events:write"]
```

```
}
// → { key: "sk_live_...", keyId, prefix }  ← save key now, shown once
```

> **IMPORTANT**
>
> Store the returned API key securely in your server-side environment variables. It cannot be retrieved again — only rotated.

5 SDK Integration

Web SDK for browser fingerprinting + server-side event submission

Installation

```
npm install @signalrisk/web-sdk
# or
yarn add @signalrisk/web-sdk
```

Browser — Device Fingerprinting

Initialise the SDK on page load. It collects device signals client-side and returns a `deviceToken` to include in your server-side event submission.

```
import { SignalRisk } from '@signalrisk/web-sdk';

// Initialise once at app startup
const sr = new SignalRisk({
  merchantId: 'merch_abc123',
  environment: 'production'
});

// Before checkout — collect fingerprint
const { deviceToken, sessionId } = await sr.collect();

// Send deviceToken + sessionId to your backend
```

Server-Side — Event Submission

Submit events from your backend using the API key. Never expose API keys in frontend code.

```
// Node.js example
const response = await fetch('https://api.signalrisk.io/v1/events', {
  method: 'POST',
  headers: {
    'Authorization': 'Bearer sk_live_your_key_here',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    eventId:    'evt_unique_id_prevent_duplicates',
    merchantId: 'merch_abc123',
    entityId:   'user_456',           // userId or deviceId
    eventType:  'payment',
    deviceToken: deviceToken,            // from browser SDK
    sessionId:   sessionId,
    payload: {
      amount:   99.99,
      currency: 'USD',
      phone:    '+1234567890',
      ip:       req.ip
    }
  })
});

const { decision, riskScore, decisionId } = await response.json();
// decision: 'ALLOW' | 'REVIEW' | 'BLOCK'
// riskScore: 0-100
```

> **IDEMPOTENCY**
>
> The `eventId` field is used for deduplication. Submit a UUID per transaction. If the same `eventId` is submitted twice within 24 hours, the original decision is returned without re-running the pipeline. This ensures safe retries from unreliable network conditions.

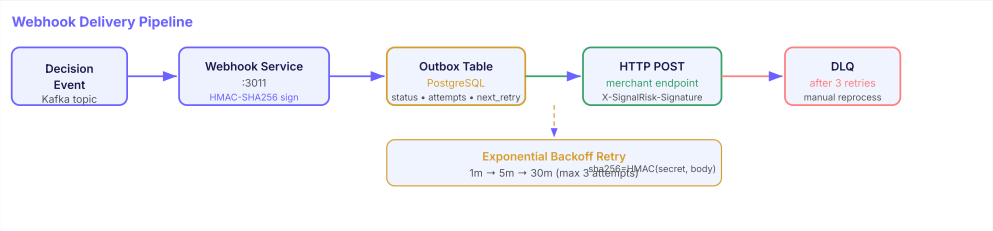
Handling the Decision

```
switch (decision) {
  case 'ALLOW':
    // Proceed with payment
    break;
  case 'REVIEW':
    // Proceed but flag for manual review
    // A case is automatically created in the analyst dashboard
```

```
    break;
  case 'BLOCK':
    // Decline the transaction
    // Return error to user
    break;
}
```

6 Webhook Integration

Receive asynchronous decision notifications with HMAC signature verification



Webhook delivery pipeline: Kafka consumer → HMAC sign → HTTP POST to merchant endpoint → exponential retry → DLQ

Registering a Webhook Endpoint

```
POST /merchants/{merchantId}/webhooks
Authorization: Bearer <accessToken>
{
  "url":    "https://your-server.com/webhooks/signalrisk",
  "events": ["decision.completed", "case.resolved"],
  "secret": "your_webhook_secret_min_32_chars"
}
```

Verifying Webhook Signatures

Every delivery includes an `X-SignalRisk-Signature` header. Always verify this before processing the payload.

```
import crypto from 'crypto';

function verifyWebhook(rawBody, signature, secret) {
  const expected = 'sha256=' + crypto
    .createHmac('sha256', secret)
    .update(rawBody)
    .digest('hex');
  return crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(expected)
  );
}

// Express handler
app.post('/webhooks/signalrisk', express.raw({ type: '*/*' }), (req, res) => {
  const sig = req.headers['x-signalrisk-signature'];
  if (!verifyWebhook(req.body, sig, process.env.WEBHOOK_SECRET)) {
    return res.status(401).end();
  }
  const event = JSON.parse(req.body);
  // process event.type, event.data...
  res.status(200).end(); // respond 200 quickly
});
```

Webhook Event Payload

```
{
  "type":      "decision.completed",
  "timestamp": "2026-03-06T10:00:00.000Z",
  "data": {
    "decisionId": "dec_xyz",
    "eventId":    "evt_abc",
    "entityId":   "user_456",
    "decision":   "BLOCK",
    "riskScore":  87,
    "reasons":   ["device_emulator", "high_velocity"],
    "latencyMs":  42
  }
}
```

> **RETRY POLICY**

> If your endpoint returns non-2xx, SignalRisk retries with exponential backoff: 1 minute, then 5 minutes, then 30 minutes. After 3 failures the event moves to the Dead Letter Queue. Always respond with 200 as quickly as possible — process asynchronously if needed.

7 API Reference

Core endpoints for event submission, case management, and rule configuration

Event Submission

Method	Path	Auth	Description
POST	/v1/events	API Key	Submit an event for fraud scoring. Returns decision + risk score synchronously.
GET	/v1/events/{id}	JWT	Retrieve a previously submitted event with its decision.

Case Management

Method	Path	Description
GET	/v1/cases	List cases with pagination. Filter by status, priority, entityId.
GET	/v1/cases/{id}	Get case detail including evidenceTimeline and riskFactors.
PUT	/v1/cases/{id}/resolve	Resolve a case: `{ resolution: "FRAUD"|"LEGITIMATE"|"INCONCLUSIVE", notes? }`

PUT	/v1/cases/{id}/escalate	Escalate to senior analyst queue.
PUT	/v1/cases/{id}/claim	Assign case to authenticated user (Fraud Ops workflow).
GET	/v1/cases/stats	Today's labeling stats: labeled, fraud, fp, accuracy, pending.

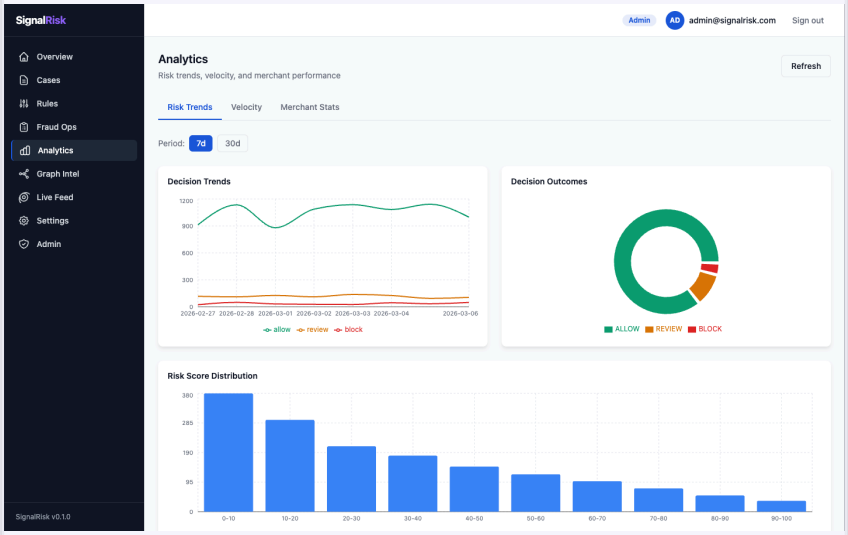
Rule Management

Method	Path	Description
GET	/v1/rules	List all rules with weights and active status.
POST	/v1/rules	Create a new rule: name, expression, outcome, weight, isActive.
PUT	/v1/rules/{id}	Update rule fields. Changes take effect within seconds.
DELETE	/v1/rules/{id}	Permanently delete a rule (consider deactivating first).

Analytics

Method	Path	Description
GET	/v1/analytics/velocity	Hourly event volume for the last 24 hours.
GET	/v1/analytics/merchants	Per-merchant: eventVolume, blockRate, avgRiskScore.
GET	/v1/analytics/kpi	Current-hour KPIs: decisionsPerHour, blockRatePct,

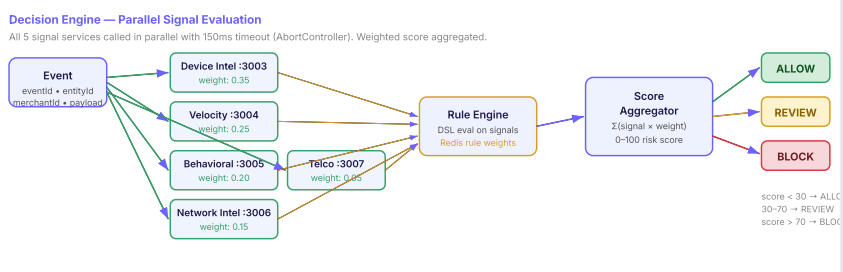
```
reviewRatePct,
avgLatencyMs.
```



Analytics page — velocity chart and merchant breakdown powered by `/v1/analytics` *endpoints*

8 Decision Engine & Rule DSL

How scores are calculated and how to author rules



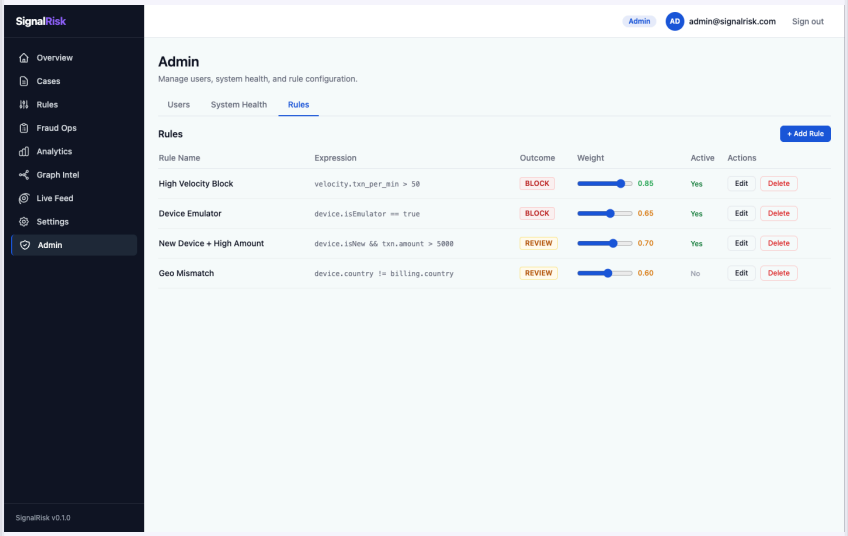
Decision Engine: all 5 signal services called in parallel with 150ms AbortController timeout, then rule evaluation and weighted score aggregation

Score Thresholds

Score Range	Decision	Default Action
0 – 29	ALLOW	Transaction proceeds normally
30 – 69	REVIEW	Transaction proceeds; case created for analyst review
70 – 100	BLOCK	Transaction declined; case created with 4h SLA

Thresholds are configurable per merchant via the Admin panel.

Rule DSL Reference



Rules management — DSL expressions, outcome badges, weight sliders, and active toggles

Available Signal Contexts

Context	Field	Type	Description
`device.`	`country`	string	ISO 3166-1 alpha-2 country code from device IP
	`isEmulator`	boolean	True if device appears to be an emulator/virtual machine
	`trustScore`	number 0–1	Composite device trust score from fingerprint analysis

velocity.	count_1h	number	Number of events from this entity in the last 1 hour
	count_24h	number	Number of events from this entity in the last 24 hours
	amount_1h	number	Total transaction amount from this entity in the last 1 hour
network.	vpnDetected	boolean	True if IP is associated with a known VPN provider
	isProxy	boolean	True if IP is a known proxy or Tor exit node
	country	string	Country resolved from IP geolocation
telco.	isValid	boolean	True if phone number passes MSISDN validation
	carrierRisk	number 0–1	Risk score for the phone carrier (high for VoIP/prepaid)
txn.	amount	number	Transaction amount in the submitted currency
	currency	string	ISO 4217 currency code (e.g. USD, GBP)

Operators

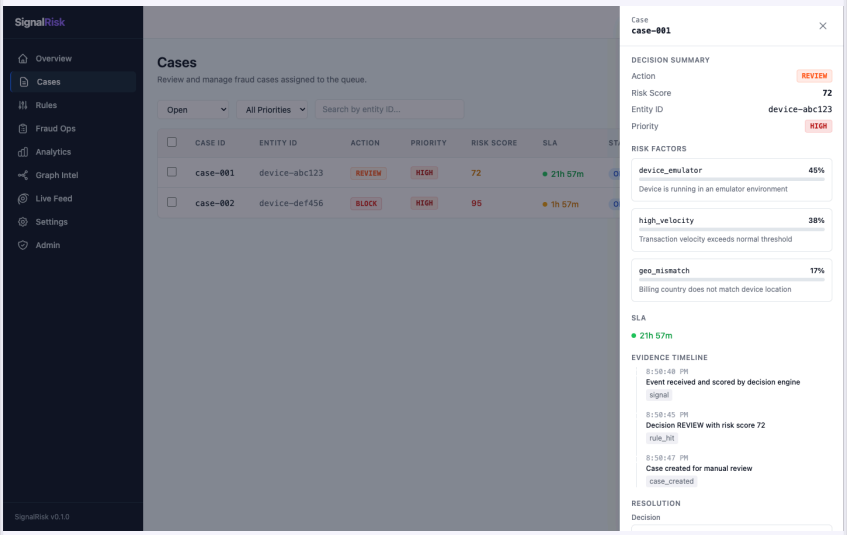
`==` `≠` `>` `<` `≥` `≤` `&&` `||` `!` `()`

Example Rules

```
// Block emulator traffic above a threshold amount
device.isEmulator == true && txn.amount > 100

// Flag high-velocity entities for review
velocity.count_1h > 15 || velocity.count_24h > 50

// Block high-risk countries with VPN
device.country == 'NG' && network.vpnDetected == true

// Review suspicious carrier + large transaction
telco.carrierRisk > 0.8 && txn.amount > 500

// Block untrusted devices from risky networks
device.trustScore < 0.2 && network.isProxy == true
```

9 Feedback Loop & Rule Weight Tuning

How analyst labels automatically improve fraud detection accuracy over time



Case detail panel — risk factor contribution bars, evidence timeline, and resolution form that feeds the weight adjustment loop

Automatic Weight Adjustment

Every case label submitted in the Fraud Ops page triggers an automatic adjustment to the rule weights stored in Redis. This creates a self-improving feedback loop — rules that correctly identify fraud become stronger over time, while rules that over-fire become weaker.

Label	Effect	Redis key
Fraud Confirmed	All matching rules: weight += 0.05 (capped at 1.0)	`rule:weight:{ruleId}`
False Positive	All matching rules: weight -= 0.03 (floor 0.1)	`rule:weight:{ruleId}`
Inconclusive	No change	—

Chargeback Integration

When a chargeback is received via the `POST /v1/chargebacks` endpoint, the system automatically labels the corresponding event as Fraud Confirmed and applies the same weight adjustment. This closes the loop between payment disputes and rule accuracy without requiring manual analyst action.

```
POST /v1/chargebacks
Authorization: Bearer sk_live_...
{
  "eventId":    "evt_abc123",
  "reason":     "fraud",
  "amount":     99.99,
  "chargebackId": "chb_xyz"
}
```

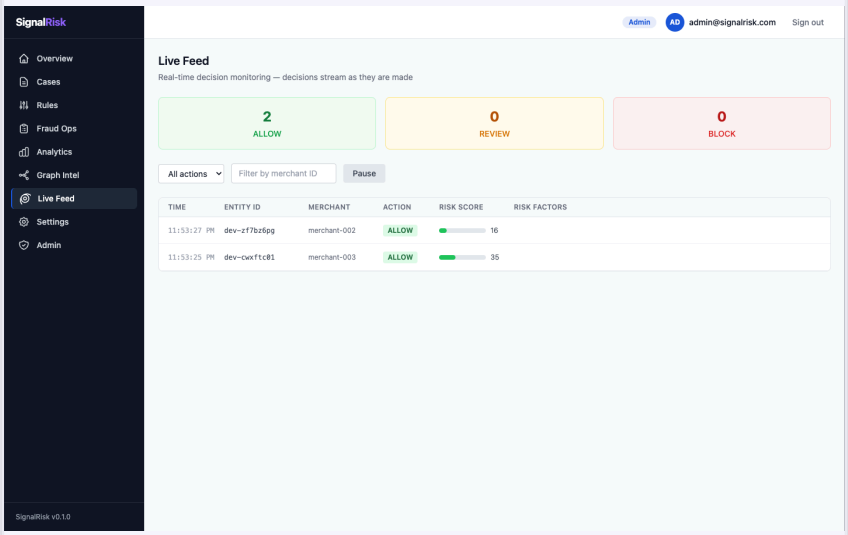
Weight Management Best Practices

- New rules should start at **0.5 weight** — let 7+ days of labels adjust automatically

- Only set weight to **1.0** for rules with verified zero false positives
- Weights below **0.2** after 30 days of labeling may indicate a rule that should be redesigned
- Monitor weight drift on the Rules page dashboard to catch rules going stale

10 Observability & Error Tracking

Tracing, metrics, logs, and real-time monitoring dashboards



Live Feed — real-time decision stream, useful for live monitoring and spotting anomalies in decision distribution

Distributed Tracing (OpenTelemetry)

Every service exports OpenTelemetry traces. Traces are correlated across all services using a `traceId` propagated through Kafka message headers and HTTP `traceparent` headers.

Service	Trace spans

event-collector	http.request, kafka.produce, rate_limit.check, api_key.validate
decision-service	signals.parallel_fetch, rule_engine.evaluate, decision.write, kafka.produce
signal services	signal.fetch, cache.get, cache.set, db.query
case-service	case.create, sla.check, case.resolve
webhook-service	webhook.deliver, webhook.retry, webhook.dlq

Prometheus Metrics

All services expose `GET /metrics` in Prometheus format. Key metrics to monitor:

Metric	Type	Alert threshold
`decision_latency_ms`	Histogram	P99 > 100ms
`signal_timeout_total`	Counter	> 1% of decisions
`decision_block_rate`	Gauge	> 15% sustained
`kafka_consumer_lag`	Gauge	> 10,000 messages
`webhook_dlq_size`	Gauge	> 0 (any DLQ events)
`rate_limit_rejections_total`	Counter	Spike > 1000/min
`sla_breached_cases_total`	Counter	Any increase

Structured Logging

All services use JSON structured logs with `traceId`, `merchantId`, `eventId`, and `decisionId` fields. Log levels: ERROR for actionable failures, WARN for degraded paths, INFO for key lifecycle events.

```
{
  "level":      "ERROR",
  "service":    "webhook-service",
  "traceId":    "abc123",
  "merchantId": "merch_xyz",
  "event":      "webhook.delivery_failed",
  "attempt":    3,
  "url":        "https://merchant.com/webhook",
  "statusCode": 503,
  "nextAction": "dlq",
  "timestamp":  "2026-03-06T10:00:00Z"
}
```

Health Endpoints

Every service exposes `GET /health` returning JSON with `status`, `version`, and dependency health checks (DB, Redis, Kafka). Used by Kubernetes liveness and readiness probes, and aggregated in the Admin → System Health dashboard.

11 Deployment Architecture

Kubernetes, Helm chart, and GitOps pipeline

Helm Chart

A full Helm umbrella chart at `deploy/helm/signalrisk/` covers all 13 services. Each service is a subchart with configurable replicas, resource limits, HPA, and health probes.

```
# Install to staging
helm upgrade --install signalrisk deploy/helm/signalrisk/
--namespace signalrisk-staging   --values
deploy/helm/signalrisk/values-staging.yaml

# Install to production
helm upgrade --install signalrisk deploy/helm/signalrisk/
--namespace signalrisk-prod   --values
deploy/helm/signalrisk/values-prod.yaml
```

Resource Allocation (Production)

Service Tier	Services	Replicas	CPU Request	Memory
Critical path	event-collector, decision-service	3–10 (HPA)	500m	512Mi
Signal services	device-intel, velocity, behavioral, network-intel, telco-intel	2–6 (HPA)	200m	256Mi

Supporting	case-service, webhook-service, rule-engine, graph-intel	2–4 (HPA)	200m	256Mi
Auth	auth-service	2–4 (HPA)	200m	256Mi

GitOps with ArgoCD

ArgoCD application-of-apps configuration at `deploy/argocd/` :

- **Staging**: auto-sync enabled — merges to `main` deploy automatically
- **Production**: manual approval required — sync triggered via ArgoCD UI or CLI after review
- Rollback: `argocd app rollback signalrisk-prod <revision>`

> **BLUE-GREEN DEPLOYS**
> Use Kubernetes `RollingUpdate` strategy with `maxSurge: 1` and `maxUnavailable: 0` for zero-downtime deployments. The event-collector and decision-service are the most critical — deploy these last in the release sequence.

12 Security Model

Authentication, authorisation, tenant isolation, and rate limiting

API Key Security

- Format: `sk_live_<32 hex chars>`
- Only bcrypt hash stored in database — plaintext never persisted
- First 8 characters used as a lookup prefix
- Rotation: generate new key, delete old — no downtime

JWT Security

- Algorithm: RS256 (asymmetric) — private key signs, public key verifies
- Access token TTL: 15 minutes
- Refresh token TTL: 7 days
- Logout: JWT added to Redis denylist until expiry

Rate Limiting

Redis Lua atomic script enforces limits at the Event Collector. On breach, returns HTTP 429 with `Retry-After` header.

```
-- Redis key: rate:{merchantId}:{endpoint}
-- Default: 1000 req/min per merchant
-- Token bucket algorithm with atomic Lua script
```

Tenant Isolation (PostgreSQL RLS)

```sql
-- Every query is preceded by:
SELECT set_config('app.merchant_id', merchantId, true);

-- RLS policy on all tables:
CREATE POLICY tenant_isolation ON cases
  USING (merchant_id = current_setting('app.merchant_id'));
```

SECURITY NOTE — KNOWN GAPS FOR PRODUCTION

1. API key validation in event-collector is currently stubbed — any non-empty string is accepted. Must be replaced with real bcrypt lookup before production.

2. Refresh token store uses in-memory Map — tokens are lost on service restart. Replace with Redis-backed store for production.

3. Outbox relay missing FOR UPDATE SKIP LOCKED — risk of duplicate webhook delivery under concurrent workers.

13 Performance SLAs & Capacity

Latency targets, throughput benchmarks, and scaling guidance

Metric	Target	Measured at
Event submission P50 latency	< 50ms	event-collector response
Decision P99 latency	< 100ms	decision-service end-to-end
Signal timeout	150ms	AbortController per signal call
Sustained throughput	5,000 req/sec	k6 load test, 3 replicas each
Webhook delivery P99	< 5 seconds	First attempt delivery
Case SLA — BLOCK	4 hours analyst resolution	Monitored every 5 min
Case SLA — REVIEW	24 hours analyst resolution	Monitored every 5 min
API uptime SLA	99.9% monthly	Excluding planned maintenance

Scaling Guidance

Horizontal Scaling

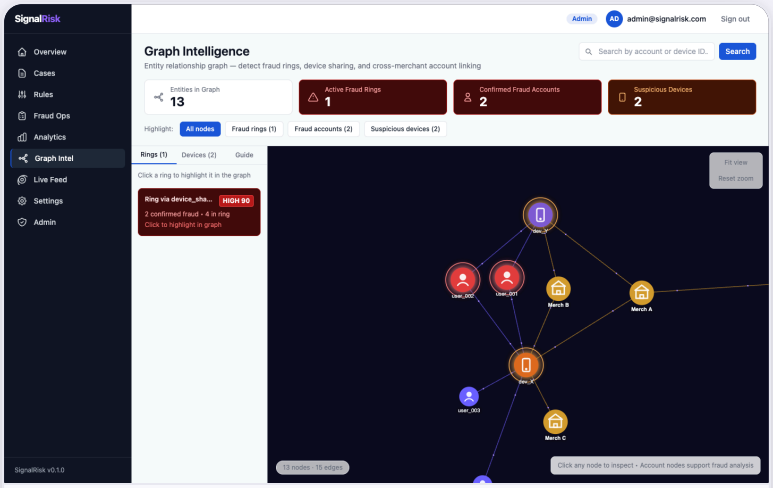
All services are stateless except for the Redis-backed rate limiter and velocity counters. Increase HPA `maxReplicas` in the Helm values file. The event-collector and decision-service are the primary scaling targets.

Kafka Partitioning

Increase partitions on `events.raw` and `events.decisions` topics to match the number of decision-service replicas. Target: 1 partition per replica for maximum parallelism.

14 Graph Intelligence Service

Neo4j-powered entity relationship graph for fraud ring detection



Graph Intelligence page — force-directed entity graph with KPI cards, fraud ring alerts, and node type filtering

Architecture Overview

The `graph-intel-service` (port 3012) maintains a Neo4j property graph of accounts, devices, and merchants. It is populated via two paths:

Path	Trigger	Relationship Written
Direct analyze API	`POST /v1/graph/analyze`	`Account -[:USES_DEVICE]→ Device`

Kafka consumer	`events.decisions` topic	`Device -[:USED_BY]→ Merchant`

Neo4j Data Model

```
// Node labels
(:Account {accountId, merchantId, isFraud,
riskScore})
(:Device  {deviceId, isEmulator, trustScore,
isSuspicious})
(:Merchant {merchantId, name})

// Relationships
(:Account)-[:USES_DEVICE]→(:Device)   // account
used this device
(:Device)-[:USED_BY]→(:Merchant)       // device
appeared at this merchant
(:Account)-[:USES_IP]→(:IPAddress)    // shared IP
signal
```

Fraud Ring Detection

A fraud ring is identified when multiple accounts share the same device and at least one has `isFraud=true`. The Cypher query traverses `USES_DEVICE` relationships:

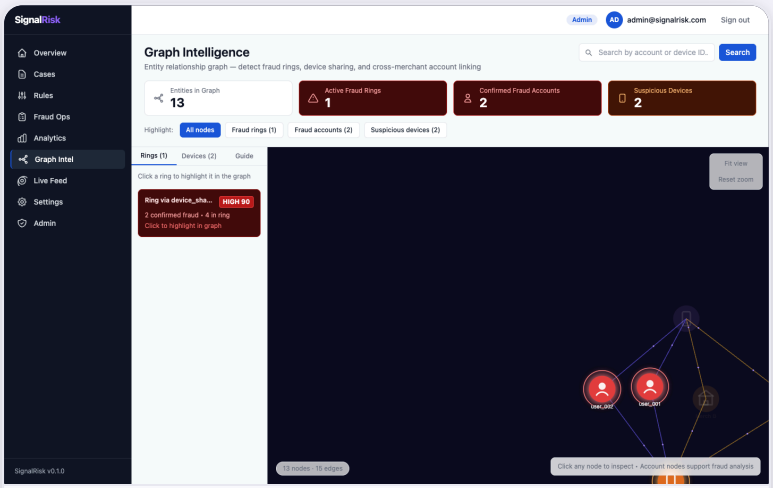
```
MATCH (a:Account)-[:USES_DEVICE]→(d:Device)←
[:USES_DEVICE]-(other:Account)
WHERE a.isFraud = true AND a.accountId <>
other.accountId
RETURN d.deviceId AS deviceId,
       collect(DISTINCT a.accountId) AS
fraudAccounts,
       collect(DISTINCT other.accountId) AS
allAccounts,
       size(collect(DISTINCT other.accountId)) * 25
+ 10 AS riskScore
```

Suspicious Device Detection

Devices shared across multiple merchants are flagged — a strong signal for synthetic account farms or device spoofing:

```
MATCH (d:Device)-[:USED_BY]→(m:Merchant)
WITH d, collect(DISTINCT m.merchantId) AS merchants
WHERE size(merchants) ≥ 2
RETURN d.deviceId, merchants, size(merchants) AS
sharingCount
```

Fraud Ring Highlighted



Fraud ring highlighted — clicking a ring card focuses the camera and dims non-member nodes to 12% opacity

Account Risk Analysis

The `POST /v1/graph/analyze` endpoint runs 4 parallel Cypher queries for a given `accountId`:

Query	Returns
Fraud neighbor count	Accounts directly connected via shared device that have `isFraud=true`
Shared device count	Accounts sharing any device with the target account
Shared IP count	Accounts sharing any IP address
Ring membership	Boolean — is this account in a detected fraud ring

Dashboard Integration

The `GET /v1/graph/summary` endpoint returns all nodes, edges, fraud rings, and suspicious devices in a single response. The dashboard renders this as a force-directed graph using `react-force-graph-2d` (D3 force simulation, canvas rendering). Each node type uses a distinct icon (person / phone / storefront) for quick visual identification.

Configuration

Environment Variable	Default	Description
`NEO4J_URI`	`bolt://localhost:7687`	Neo4j bolt connection
`NEO4J_USERNAME`	`neo4j`	Database user
`NEO4J_PASSWORD`	`password`	Database password (rotate in production)

15 Go-Live Checklist

Required steps before deploying SignalRisk to production

Security (Must complete before production)

1. Replace API key validation stub in `event-collector` with real bcrypt lookup against database
2. Replace in-memory refresh token Map with Redis-backed store for persistence across restarts
3. Add `FOR UPDATE SKIP LOCKED` to outbox relay to prevent duplicate webhook delivery
4. Rotate all development JWT key pairs — generate new RS256 keypair for production
5. Set `ALLOWED_ORIGINS` environment variable to production dashboard domain

Infrastructure

6. Run database migrations on production PostgreSQL with RLS policies verified

7. Configure Redis cluster with AOF persistence enabled
8. Create Kafka topics with correct partition counts for expected throughput
9. Deploy Neo4j with transaction-level durability settings
10. Verify all 13 health endpoints return `{"status":"ok"}`

Monitoring

11. Configure Prometheus scrape targets for all 13 `/metrics` endpoints
12. Import SignalRisk Grafana dashboards from `deploy/grafana/`
13. Set up PagerDuty / Opsgenie alerts for: P99 latency > 100ms, block rate > 15%, DLQ > 0, any service down
14. Configure OpenTelemetry collector to export traces to Jaeger or Tempo

Integration Testing

15. Submit 100 test events via the SDK in staging — verify decisions are returned in < 100ms

16. Trigger a webhook delivery and verify HMAC signature verification succeeds on your endpoint

17. Test chargeback endpoint — confirm matching case is auto-labeled and rule weights adjust

18. Run k6 load test at 20% of target traffic — confirm P99 < 100ms and no errors

Analyst Onboarding

19. Create analyst accounts via Admin → Users and share login credentials securely

20. Configure initial rule set via Admin → Rules with conservative weights (0.5)

21. Verify Live Feed WebSocket connects and shows real-time events after first production submission

> **RECOMMENDED GO-LIVE SEQUENCE**
>
> Deploy data stores (PostgreSQL, Redis, Kafka, Neo4j) → deploy auth-service → deploy signal services → deploy rule-engine + decision-service → deploy event-collector → deploy case + webhook services → deploy dashboard → smoke test → enable production traffic gradually (10% → 50% → 100%).